01 / PROPÓSITO Y ALCANCE

Práctica 2 Streams paralelos

Solución implementada, marco teórico y preparación para la sustentación.

Universidad Nacional de Colombia. Computación Paralela y Distribuida. Ejercicio 2. Fecha de preparación: 9 de noviembre de 2026.

Qué quedó resuelto

Consulta	Selección	Resultado
Promedio de edad	Estudiantes activos	double; NaN si no hay activos
Nombre más frecuente	Estudiantes inactivos	String; null si no hay inactivos
Cantidad de reprobados	Inactivos, edad > 20 y nota < 65	int; 0 si nadie cumple

La implementación conserva las seis firmas públicas, mantiene los tres métodos imperativos y usa streams paralelos en las tres consultas solicitadas. No introduce ciclos for, while o do en sus cuerpos.

Cómo recorrer este informe

- Páginas 2 a 6: conceptos, implementación real y justificación de las operaciones.
- Página 7: un conjunto de estudiantes resuelto a mano, de principio a fin.
- Páginas 8 y 9: complejidad, rendimiento y evidencia de verificación.
- Páginas 10 a 12: preguntas con respuestas, ejercicios y fuentes primarias.
- Páginas 13 a 16: composición de los tests, aserciones y criterios de aprobación.

Archivos que debes reconocer

`ejercicio_2/src/main/java/co/edu/unal/paralela/StudentAnalytics.java` contiene la solución. `verificacion/` contiene pruebas adicionales y el registro de ejecución. La guía original y las pruebas proporcionadas se conservaron.

Alcance de los datos: arreglos y estudiantes no nulos, nombres no nulos y edades finitas del dominio académico. No se añadió una capa de validación de datos.

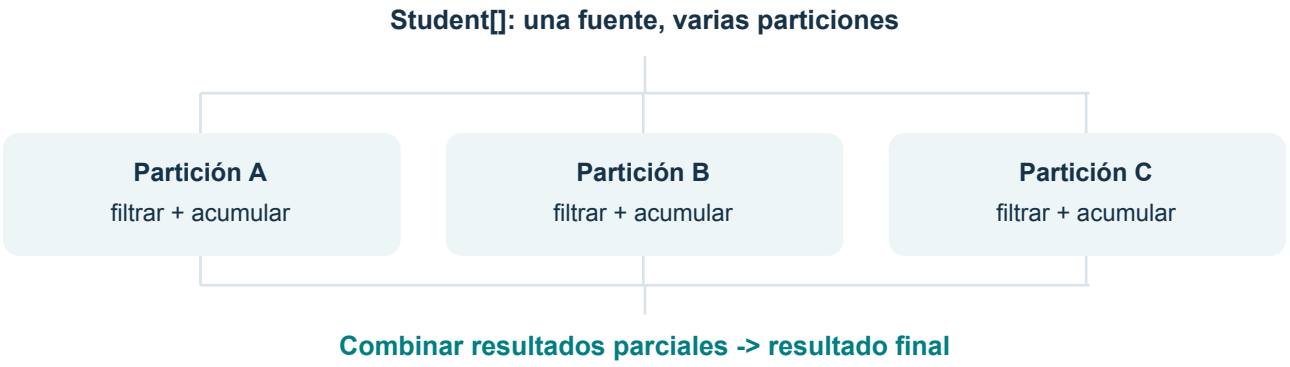
02 / MARCO TEÓRICO

Del ciclo al pipeline

Un stream describe qué calcular sobre una secuencia de datos.

Una colección o un arreglo almacena datos. Un stream organiza su recorrido. Puede consumir la misma fuente sin modificarla, pero cada stream se utiliza una sola vez. [1, 2]

Elemento	Papel en esta práctica
Fuente	`Stream.of(studentArray)` produce un Stream<Student>.
Modo de ejecución	`.parallel()` habilita la ejecución paralela del pipeline.
Operación intermedia	`filter` conserva elementos; `mapToDouble` extrae edades.
Operación terminal	`average`, `collect`, `max` o `count` produce un resultado.
Evaluación diferida	La selección y transformación se recorren al iniciar la terminal.



Qué significa ejecutar en paralelo

El arreglo permite dividir rangos de índices con bajo costo. En OpenJDK, los streams paralelos se ejecutan habitualmente mediante Fork/Join: tareas procesan particiones y se combinan sus resultados. No se crea un hilo por estudiante ni se promete un número fijo de hilos. El hilo que invoca también puede participar. [7, 8]

Paralelismo significa que parte del trabajo puede ocurrir simultáneamente; concurrencia significa que varias tareas progresan durante el mismo intervalo. Aquí se aplica paralelismo de datos: la consulta se repite sobre diferentes subconjuntos.

Condiciones para una reducción correcta

Las funciones deben evitar interferir con la fuente y depender de estado compartido cambiante. Las reducciones deben tener una combinación compatible con su acumulación y con una identidad. La suma de conteos satisface estas condiciones. [2, 3]

03 / IMPLEMENTACIÓN 1

Promedio de edad de activos

Filtrar estudiantes, extraer edades y reducir a una media.

```
public double averageAgeOfEnrolledStudentsParallelStream(
        final Student[] studentArray) {
    return Stream.of(studentArray).parallel()
            .filter(Student::checkIsCurrent)
            .mapToDouble(Student::getAge)
            .average()
            // La versión imperativa produce 0.0 / 0.0 si no hay activos.
            .orElse(Double.NaN);
}
```

Por qué cada operación está ahí

- `filter(Student::checkIsCurrent)` conserva únicamente los activos. La referencia a método equivale a la lambda `s -> s.checkIsCurrent()`.
- `mapToDouble(Student::getAge)` conserva edades fraccionarias y produce un DoubleStream, evitando envolver cada edad en un objeto Double.
- `average()` entrega un OptionalDouble. `orElse(Double.NaN)` conserva el caso vacío de la versión imperativa: la división 0.0 / 0.0 produce NaN. [4]

El promedio no se combina promediando promedios

Sean A los estudiantes activos. La consulta calcula suma(edad de s en A) / |A|. Un resultado parcial útil es el par (suma, cantidad), cuya combinación es (S1 + S2, C1 + C2). Solo al final se divide la suma entre la cantidad total.

Partición	Suma	Cantidad	Promedio local
A: edades 18 y 22	40	2	20
B: edad 24	24	1	24
Combinación correcta	64	3	21,333333...

Promediar 20 y 24 daría 22: es incorrecto porque ambas particiones tienen diferente cantidad de estudiantes. `average()` gestiona la acumulación y combinación por nosotros.

Precisión numérica

La suma en coma flotante depende del orden y no es exactamente asociativa. La API permite diferencias de redondeo entre recorridos. Por eso se compara con tolerancia; la prueba original usa error absoluto menor que 1E-5. NaN se comprueba con `Double.isNaN`, nunca con `valor == Double.NaN`. [4]

04 / IMPLEMENTACIÓN 2

Nombre más frecuente

La reducción construye frecuencias por partición y suma los mapas.

```
public String mostCommonFirstNameOfInactiveStudentsParallelStream(
        final Student[] studentArray) {
    final Map<String, int[]> nameCounts = Stream.of(studentArray).parallel()
            .filter(s -> !s.checkIsCurrent())
            // collect aísla los mapas por partición. int[1] permite
            // incrementar sin crear un Integer nuevo por estudiante.
            .collect(HashMap<String, int[]>::new,
                    (counts, student) -> {
                        final String name = student.getFirstName();
                        final int[] count = counts.get(name);
                        if (count == null) {
                            counts.put(name, new int[] {1});
                        } else {
                            count[0]++;
                        }
                    },
                    (left, right) -> right.forEach((name, frequency) -> {
                        final int[] count = left.get(name);
                        if (count == null) {
                            left.put(name, frequency);
                        } else {
                            count[0] += frequency[0];
                        }
                    }));

    // El trabajo sobre los estudiantes es paralelo. El máximo se busca
    // sobre los pocos nombres distintos; HashMap no define desempates.
    return nameCounts.entrySet().stream()
            .max(Comparator.comparingInt(entry -> entry.getValue()[0]))
            .map(Map.Entry::getKey)
            .orElse(null);
}
```

El mapa asocia cada nombre con un arreglo de una posición, `int[1]`. Esa celda contiene su frecuencia. Usar un contador primitivo mutable evita crear un Integer nuevo en cada incremento. La modificación se limita al resultado parcial de la reducción; los estudiantes y su arreglo no cambian.

05 / REDUCCIÓN Y SEGURIDAD

Cómo se cuentan los nombres

La seguridad está en el aislamiento de los acumuladores, no en HashMap.

Parte de collect	Qué hace la implementación
Supplier	`HashMap<String, int[]>::new` crea un mapa vacío para una partición.
Accumulator	Busca el nombre una vez. Si es nuevo, inicia su contador en 1; si existe, incrementa la celda.
Combiner	Recorre el mapa derecho y suma sus frecuencias al izquierdo. Si una clave no existía, incorpora su contador.
Selección final	Recorre las entradas del mapa combinado y toma la mayor frecuencia con `max`. Sin entradas, devuelve null.

Invariante que explica la corrección

Después de procesar una partición P, el contador de cada nombre x es exactamente la cantidad de estudiantes inactivos de P cuyo nombre es x. Para dos particiones disjuntas, la frecuencia combinada es f(P1, x) + f(P2, x). El mapa vacío funciona como identidad y la suma de frecuencias es asociativa. [3]

Ejemplo: {Ana: 2, Luis: 1} combinado con {Ana: 1, Eva: 2} produce {Ana: 3, Luis: 1, Eva: 2}. El máximo por frecuencia es Ana. No se debe calcular la moda global eligiendo simplemente una de las modas locales.

Por qué no hay una carrera de datos

La reducción de tres argumentos usa contenedores separados y los combina de forma controlada. El mismo mapa o contador parcial no se incrementa simultáneamente desde tareas independientes. HashMap e int[] no son seguros para escritura concurrente arbitraria; usarlos en un `forEach` paralelo con un mapa externo compartido sí introduciría un problema. [2, 3]

Empates y alternativas

El imperativo recorre un HashMap, que no define orden de iteración. No hay una regla de desempate en la guía: se devuelve un nombre de frecuencia máxima; ante empate, ambas versiones pueden elegir nombres distintos. La comparación exacta se exige cuando hay una moda única. No se añadió un desempate alfabético. [5]

La versión declarativa habitual usa `groupingBy(Student::getFirstName, counting())`. La reducción implementada expresa ese mismo conteo con menos capas de funciones por registro. `groupingByConcurrent` puede introducir contención si millones de registros comparten pocos nombres; no garantiza más velocidad. [6]

06 / IMPLEMENTACIÓN 3

Reprobados mayores de 20

Las tres condiciones se deben cumplir al mismo tiempo.

```
public int countNumberOfFailedStudentsOlderThan20ParallelStream(
        final Student[] studentArray) {
    return (int) Stream.of(studentArray).parallel()
            .filter(s -> !s.checkIsCurrent()
                    && s.getAge() > 20 && s.getGrade() < 65)
            // El conteo no supera la longitud del arreglo, que es int.
            .count();
}
```

La condición exacta es: inactivo Y edad estrictamente mayor que 20 Y nota estrictamente menor que 65. La definición de reprobado proviene del ejercicio; no basta una nota baja si el estudiante sigue activo.

Activo	Edad	Nota	¿Se cuenta?	Razón
No	21	64	Sí	Cumple las tres condiciones
No	20	64	No	20 no es mayor que 20
No	20,01	64	Sí	La edad es double
No	25	65	No	65 no es menor que 65
Sí	25	10	No	Todavía está activo
No	19	40	No	No supera la edad exigida

Por qué count es suficiente

Después del filtro, cada elemento representa exactamente un estudiante que debe contarse. No hace falta crear una lista de resultados ni mapear cada uno a 1 manualmente. Si ninguna fila cumple, el conteo es 0.

Por qué se puede convertir long a int

`count()` devuelve long por contrato de Stream. Aquí la fuente es un único Student[]: el resultado no puede superar `studentArray.length`, que es int. La conversión respeta la firma original sin perder información. Para otra fuente potencialmente mayor, esta justificación no sería válida. [2]

Identidad y combinación

Cada partición produce un conteo c. El estado inicial es 0 y la combinación es c1 + c2. Como los subconjuntos del arreglo son disjuntos y los conteos no superan su longitud total, no hay duplicación ni desbordamiento del resultado.

07 / EJEMPLO TRABAJADO

Ocho estudiantes, tres consultas

Resuelve primero a mano y luego identifica cada etapa del pipeline.

#	Nombre	Edad	Nota	Activo
1	Ana	18	90	Sí
2	Luis	22	60	Sí
3	Ana	21	64	No
4	Ana	20	40	No
5	Luis	25	65	No
6	Eva	30	50	No
7	Ana	23	60	No
8	Luz	24	10	Sí

1. Promedio de activos

Se conservan las filas 1, 2 y 8. Sus edades son 18, 22 y 24. Suma = 64; cantidad = 3; promedio = 64 / 3 = 21,333333... años. Las notas 60 y 10 de los activos no influyen en esta consulta.

2. Nombre más frecuente entre inactivos

Se conservan las filas 3, 4, 5, 6 y 7. Sus nombres son Ana, Ana, Luis, Eva y Ana. Frecuencias: Ana = 3, Luis = 1, Eva = 1. El resultado es Ana. Los nombres de las filas activas no intervienen.

3. Reprobados mayores de 20

Cumplen las filas 3, 6 y 7: el resultado es 3. La fila 4 se excluye por tener exactamente 20 años; la 5, por nota 65. Las filas 2 y 8 se excluyen por estar activas, aunque tengan más de 20 años y nota inferior a 65.

Una partición posible

Dividiendo después de la fila 4, los parciales del promedio son (40, 2) y (24, 1). Los conteos de reprobados son 1 y 2. Los mapas de nombres inactivos son {Ana: 2} y {Luis: 1, Eva: 1, Ana: 1}. Combinar estos estados produce los tres resultados anteriores; la JVM puede elegir otra división.

08 / COMPLEJIDAD Y DESEMPEÑO

Paralelo no implica más rápido

La distribución del trabajo tiene costos y debe medirse.

Sea n la cantidad de estudiantes, k la cantidad de nombres distintos entre inactivos y q la cantidad de estados parciales que la ejecución crea. q no tiene por qué ser igual al número de núcleos.

Consulta	Trabajo y estado auxiliar
Promedio	Recorrido O(n), estados de tamaño constante por partición. Evita la lista de activos de la versión imperativa.
Reprobados	Recorrido O(n), conteos de tamaño constante por partición.
Moda	Acumulación esperada O(n) con buen hashing y selección O(k). La fusión añade trabajo: cota gruesa O(qk). Puede mantener hasta O(qk) entradas parciales, frente a O(k) del mapa final.

Speedup, eficiencia y límite ideal

Speedup S = T_secuencial / T_paralelo. S > 1 indica aceleración; S < 1 indica ralentización. Eficiencia E = S / p, donde p es el número de recursos de procesamiento considerados. No se debe confundir p con la cantidad de particiones.

En el modelo ideal de Amdahl, S(p) = 1 / ((1 - f) + f / p), donde f es la fracción paralelizable. Por ejemplo, con f = 0,9 y p = 8, el límite ideal es aproximadamente 4,71. La creación de tareas, las fusiones y la presión de memoria pueden reducirlo aún más.

Qué puede frenar esta práctica

- Pocos registros o muy poco trabajo por registro: coordinar tareas puede costar más que recorrer el arreglo secuencialmente.
- Acceso a objetos y memoria: más hilos pueden saturar el ancho de banda; más núcleos no garantizan una reducción proporcional del tiempo.
- Muchos nombres diferentes: se crean más entradas y la fusión de mapas crece. Pocos nombres pueden favorecer mapas locales, sin escrituras compartidas.

Cómo interpretar las pruebas originales

Exigen S > 1,2 para promedio y reprobados y S >= 0,5 × procesadores para la moda. Usan currentTimeMillis, pocas repeticiones y no aíslan calentamiento JIT, recolección de basura ni carga del sistema. Son criterios del ejercicio; no constituyen una garantía universal de rendimiento.

09 / VALIDACIÓN Y REPRODUCCIÓN

Qué se comprobó

La verificación conserva las pruebas entregadas y añade casos de frontera.

Suite	Resultado de la ejecución final
Original	6 pruebas, 0 fallos, 0 ignoradas; 819 ms en total. Incluye 3 pruebas de corrección y 3 de rendimiento.
Adicional	15 pruebas, 0 fallos, 0 ignoradas; 61 ms en total. Una prueba compara 200 arreglos pseudoaleatorios.
Entorno	OpenJDK 27, macOS aarch64; 10 procesadores visibles; paralelismo del commonPool = 9. Compilación con --release 8.

Los tiempos anteriores corresponden a suites completas, incluyendo generación de datos y aserciones. No son tiempos individuales de los métodos ni permiten calcular su speedup. El registro exacto está en `verificacion/resultados.txt`.

Cobertura de las pruebas adicionales

Arreglo vacío, todos activos, todos inactivos, un solo estudiante, edades fraccionarias, edad exactamente 20, nota exactamente 65, activo con nota baja, moda solo de inactivos, mayúsculas, empates y colisiones de hash, integridad del arreglo y ausencia de estado acumulado entre llamadas.

La comparación aleatoria usa semilla 20261109, longitudes entre 0 y 3000 y edades entre 0 y 100. Promedio: tolerancia 1E-9 o NaN. Conteo: igualdad exacta. Moda: frecuencia máxima, e igualdad con el imperativo cuando el máximo es único.

Ejecutar desde practica_2

```
bash verificacion/verificar.sh
```

Requiere JDK 8 o posterior y Bash. Usa junit-4.12.jar y hamcrest-core-1.3.jar ya incluidos. Compila en una carpeta temporal, ejecuta ambas suites, elimina los .class temporales y guarda el resultado. Si alguna prueba falla, sale con código distinto de cero. No necesita descargar dependencias.

Variación observada y entorno Maven

La primera implementación con un collector estándar tuvo una ejecución con S = 4,612 para moda frente al mínimo 5,0. Ese registro se conserva en `verificacion/historial/01-collector-estandar.txt`. Tras cambiar la reducción pasaron las 21 pruebas. Esto no garantiza el mismo rendimiento en otra ejecución.

No se modificó pom.xml. El intento Maven offline no pudo abrir un archivo de bloqueo en la caché local ~/.m2. La comprobación se realizó directamente con javac y JUnit. La compatibilidad de API Java 8 se comprobó al compilar; la ejecución se hizo en Java 27, no en una JVM 8.

10 / PREGUNTAS DE SUSTENTACIÓN

Fundamentos y promedio

Preguntas probables con respuestas breves que puedes ampliar oralmente.

1. ¿Qué diferencia hay entre un arreglo y un stream?

El arreglo almacena estudiantes; el stream define su procesamiento. Filtrar no elimina estudiantes del arreglo. Cada consulta crea un stream nuevo porque el anterior queda consumido después de su operación terminal.

2. ¿Qué cambia al llamar parallel()?

Se habilita la ejecución paralela del pipeline. La JVM divide y planifica trabajo; no se garantiza un hilo por elemento ni una aceleración fija.

3. ¿Por qué filter y mapToDouble son intermedias?

Devuelven otro stream y no producen todavía el valor final. El recorrido se activa con average, collect o count. Esta evaluación diferida permite fusionar operaciones durante el procesamiento.

4. ¿Qué significa Student::getAge?

Es una referencia al método de instancia que equivale a s -> s.getAge(). mapToDouble recibe una función que transforma cada Student en un double.

5. ¿Por qué no mapToInt para las edades?

Student almacena la edad como double. Convertir a int truncaría fracciones y podría cambiar tanto el promedio como la frontera de edad 20.

6. ¿Por qué no devolver cero si no hay activos?

La referencia calcula 0.0 / 0.0 y devuelve NaN. Cero sería una edad media definida y cambiaría el comportamiento solicitado. Por eso se usa orElse(Double.NaN).

7. ¿Se pueden promediar directamente los promedios parciales?

Solo si las particiones tienen igual cantidad de elementos seleccionados. En general hay que combinar sumas y cantidades, o ponderar cada promedio por su cantidad antes de dividir por el total.

8. ¿Por qué los doubles no se comparan siempre con ==?

El orden de suma puede variar y provocar redondeo distinto. Se usa una tolerancia adecuada al dominio; NaN requiere Double.isNaN. La precisión numérica y la corrección de la selección son aspectos diferentes.

11 / PREGUNTAS DE SUSTENTACIÓN

Moda, reducción y concurrencia

Estas preguntas explican las decisiones de la implementación real.

9. ¿Cuáles son los tres argumentos de collect?

Supplier: mapa vacío. Accumulator: incorporar un estudiante incrementando su nombre. Combiner: unir dos mapas sumando frecuencias por clave. Son la creación, acumulación y combinación del estado parcial.

10. ¿Por qué el contador es int[] y no Integer?

Integer es inmutable: incrementarlo requiere obtener otro valor envuelto. int[1] ofrece una celda mutable por nombre y partición. El arreglo de una posición es un contenedor del contador, no una lista de estudiantes.

11. ¿Cómo puede ser seguro usar HashMap en paralelo?

Porque los mapas parciales están aislados y collect controla su combinación. HashMap sigue sin ser seguro para escritura concurrente arbitraria. Compartir un mapa externo entre todas las lambdas sería otro diseño y tendría carreras.

12. ¿Por qué la combinación debe ser asociativa?

La planificación puede cambiar el agrupamiento de los resultados parciales. Sumar conteos por clave conserva el resultado al reagruparlos. Restar frecuencias o elegir solo una moda parcial no cumple esa propiedad.

13. ¿Por qué no tomar la moda de las modas locales?

Se pierde información. Si A tiene Ana: 6, Eva: 5 y B tiene Luis: 6, Eva: 5, las modas locales son Ana y Luis; la moda global es Eva con 10. Se deben conservar las frecuencias de todas las claves hasta combinarlas.

14. ¿Qué pasa si hay empate o no hay inactivos?

Sin inactivos, el mapa queda vacío y se devuelve null. Con empate se devuelve una moda válida: HashMap no promete orden. Una regla alfabética requeriría acordar un contrato adicional y un comparador diferente.

15. ¿Por qué max usa un stream secuencial?

El recorrido costoso sobre los n estudiantes ya fue paralelo. El máximo recorre solo k nombres distintos; con pocos nombres, lanzar otra reducción paralela puede añadir más coordinación que trabajo útil.

16. ¿groupingByConcurrent sería siempre mejor?

No. Una acumulación compartida puede concentrar escrituras en pocas claves y generar contención. Hay que medir considerando cardinalidad, tamaño de entrada y costo de fusionar mapas locales.

12 / PREGUNTAS, EJERCICIOS Y FUENTES

Defender y extender la solución

Comprueba que puedes justificar fronteras, tipos y rendimiento.

17. ¿Un activo con nota 40 se cuenta como reprobado aquí?

No. Debe ser inactivo, tener edad > 20 y nota < 65. Tampoco se cuentan la edad exactamente 20 ni la nota exactamente 65.

18. ¿Por qué count devuelve long y el método int?

Stream admite fuentes grandes y count usa long. Esta fuente es un arreglo con longitud int, por lo que el conteo cabe en la firma exigida.

19. ¿Qué es una condición de carrera en este contexto?

Dos tareas leen el mismo contador y ambas escriben su incremento, perdiendo una actualización. Un contador externo usado desde forEach paralelo puede sufrirla; la reducción con estados aislados la evita.

20. ¿Cómo demostrarías que paralelo mejora el tiempo?

Verificaría resultados, calentaría la JVM, consumiría los resultados para evitar eliminación de trabajo, usaría alta resolución, varias mediciones y tamaños, y reportaría entorno y dispersión. Un único tiempo no basta.

Tres ejercicios rápidos y sus respuestas

A. Edades activas 18,5 y 21,5; un inactivo de 99: promedio = 20. B. Inactivos (20, 64), (21, 65), (21, 64): se cuenta 1. C. Si una prueba exige S >= 5 y mide 4,8, puede fallar por rendimiento aunque los resultados sean correctos; primero hay que distinguir ambas causas.

Fuentes y material utilizado

Guía del Ejercicio 2 y fuentes locales Student.java, StudentAnalytics.java y StudentAnalyticsTest.java. Referencias oficiales de Java 8 consultadas el 9 de noviembre de 2026:

[1] Paquete java.util.stream: pipeline, evaluación y reducción

[2] Stream: filter, collect, count y contratos de uso

[3] Collector: acumulación, aislamiento e identidad

[4] DoubleStream: average, vacío y precisión

[5] HashMap: orden y sincronización

[6] Collectors: groupingBy, counting y reducción concurrente

[7] ForkJoinPool: commonPool y work-stealing

[8] OpenJDK: tareas de streams sobre CountedCompleter

13 / TESTS: ESTRUCTURA

Anatomía de una prueba

Dos suites, 21 métodos de prueba y varias comprobaciones dentro de cada método.

Componente	Responsabilidad
StudentAnalyticsTest	6 pruebas originales: 3 de corrección y 3 de rendimiento.
StudentAnalyticsEdgeTest	15 pruebas adicionales: casos pequeños, fronteras y comparación de 200 arreglos dentro de un solo test.
RunTests	Ejecuta ambas suites con JUnitCore, reúne resultados y decide el estado global.
verificar.sh	Compila con javac, ejecuta el runner, guarda la salida y limpia las clases temporales. Usa los JAR incluidos.

Cómo se descubre y ejecuta cada test

Las dos clases heredan de TestCase y usan métodos públicos void, sin parámetros, cuyo nombre empieza por test. Es el estilo de JUnit 3, compatible con el runner de JUnit 4.12 incluido. No necesitan @Test. Los helpers generan datos o comparten lógica, pero no cuentan como tests.

Preparar, ejecutar y comprobar

Por ejemplo, la frontera de nota crea tres estudiantes inactivos de 21 años con notas 64, 65 y 66. Invoca el conteo paralelo y exige 1: solo 64 es menor que 65. Si la implementación usara <= 65, devolvería 2 y fallaría.

Aserción	Condición para cumplirla
assertEquals	Igualdad de enteros o nombres; para double se indica una tolerancia.
assertTrue	La condición debe ser verdadera: un umbral, una moda válida o isNaN.
assertNull	Se exige null cuando no hay nombre que devolver.
assertSame	Se exige la misma referencia en cada posición del arreglo.

Un test aprueba si termina sin aserciones fallidas ni excepciones no controladas. Si una aserción falla, se interrumpe ese test y no se ejecutan sus comprobaciones restantes. JUnit recoge el problema y continúa con las demás pruebas.

Los nombres completos de todos los tests, sus entradas y sus resultados esperados están documentados en `informe/tests.md`.

14 / TESTS ORIGINALES

Seis tests y sus umbrales

Las pruebas funcionales comparan resultados; las de rendimiento añaden una exigencia temporal.

Datos generados por cada helper

Se crea un arreglo de 2.000.000 estudiantes con Random(123). Los primeros 600.000 son activos y los otros 1.400.000 son inactivos. Se elige entre seis nombres y seis apellidos. Las edades están en [0, 100) y las notas son enteros entre 1 y 100, inclusive. Cada helper genera un arreglo nuevo.

Consulta	Test de corrección	Test terminado en Perf
Promedio	Error absoluto frente al imperativo < 1E-5.	La misma comprobación y S > 1,2.
Moda	Nombre exactamente igual al imperativo para el conjunto generado.	La misma comprobación y S >= 0,5 × p.
Conteo	Entero exactamente igual al imperativo.	La misma comprobación y S > 1,2.

Qué hace el helper compartido

- Calcula una referencia con el método imperativo, antes de cronometrar.
- Mide un bloque de llamadas imperativas con currentTimeMillis.
- Ejecuta una llamada paralela y comprueba su resultado con aserciones.
- Mide un bloque de llamadas paralelas y devuelve S = T_secuencial / T_paralelo.

Los tests de corrección pasan 1 repetición por bloque y no exigen un speedup. Los Perf pasan 10 repeticiones y sí comprueban el cociente. Los bloques comparan igual cantidad de llamadas; la generación de datos queda fuera de esos cronómetros.

Ejemplos de aprobación: tiempos ilustrativos

T secuencial	T paralelo	S	Decisión
120 ms	100 ms	1,2	No aprueba promedio/conteo: la desigualdad es estricta.
121 ms	100 ms	1,21	Sí cumple el umbral de promedio/conteo.
500 ms	100 ms	5	Sí cumple moda con p = 10; se admite igualdad.

p es availableProcessors(): procesadores visibles para la JVM, no necesariamente núcleos físicos. Aprobar Perf también requiere el resultado correcto: una aceleración alta por sí sola no es suficiente.

15 / TESTS ADICIONALES

Quince casos de comprobación

Las expectativas pequeñas se calculan a mano; una prueba amplía la cobertura con datos aleatorios.

Caso	Condición para aprobar
1. Arreglo vacío	Promedio NaN, nombre null y conteo 0.
2. Todos activos	Edades 18 y 24: promedio 21, nombre null y conteo 0.
3. Todos inactivos	Dos Ana de 22 y 24, notas 64 y 50: NaN, Ana y 2.
4. Edades fraccionarias	Activos 18,5 y 21,25: promedio 19,875, tolerancia 1E-12; el inactivo de 90 no participa.
5. Frontera de edad	19,99; 20; 20,0001, inactivos con nota 64: conteo 1.
6. Frontera de nota	21 años, inactivos, notas 64, 65 y 66: conteo 1.
7. Activo con nota baja	Dos de 25 años y nota 10, uno activo: conteo 1.
8. Moda solo de inactivos	Tres Ana activos; dos Eva y un Luis inactivos: Eva.
9. Mayúsculas	Un Ana y dos ana inactivos: resultado ana.
10. Empate y colisión	FB y Ea tienen frecuencia 2 y el mismo hash; Otro, 1. Se acepta FB o Ea como ganador.
11. Fuente intacta	Tras las tres consultas se conservan las referencias y el orden del arreglo, comprobados con assertSame.
12. Un activo	Ana activa, 20,5 años y nota 64: 20,5, null y 0.
13. Un inactivo	Ana inactiva, 20,5 años y nota 64: NaN, Ana y 1.
14. Llamadas repetidas	Alternar dos arreglos 12 veces mantiene sus modas Ana/null y conteos 1/0 sin acumular estado.
15. Equivalencia aleatoria	200 arreglos con semilla 20261109; longitudes 0 a 3000; edades [0, 100); notas 0 a 100.

En el caso aleatorio: promedio igual al imperativo con tolerancia 1E-9 o NaN; conteo exactamente igual. Para la moda se cuentan inactivos en un mapa secuencial independiente: el ganador debe tener frecuencia máxima; si es único, también debe coincidir con el imperativo. Doscientos arreglos forman un test, no doscientos tests.

16 / APROBACIÓN Y DIAGNÓSTICO

Cómo leer el resultado

La salida distingue una discrepancia de cálculo, un fallo temporal y un problema de ejecución.

El comando y su resultado global

```
bash verificacion/verificar.sh
```

El script crea una carpeta temporal, compila las clases de aplicación, pruebas y runner, y ejecuta RunTests. La salida se muestra y se guarda en `verificacion/resultados.txt`. Al terminar, se borran las clases temporales.

RunTests suma getFailureCount() de ambas suites. Con 0 fallos imprime RESULTADO: OK y finaliza con código 0. Con uno o más fallos imprime las trazas, RESULTADO: FALLO y sale con código 1. La opción pipefail evita que tee oculte el error del proceso que ejecuta las pruebas.

Qué revisar	Evidencia de la ejecución publicada
Suite original	6 pruebas, 0 fallos, 0 ignoradas; 819 ms.
Suite adicional	15 pruebas, 0 fallos, 0 ignoradas; 61 ms.
Estado global	RESULTADO: OK. Registro del 09.11.2026 sobre Java 27.

El runner no impone una aserción separada de exactamente 21 pruebas o cero ignoradas; decide por los fallos. Para confirmar la cobertura prevista, revisa también 6 + 15 y cero ignoradas. Los milisegundos del resumen son de suites completas, no de métodos individuales.

Interpretar un fallo

Señal	Interpretación
Valor o error numérico	Revisar selección de estudiantes, cálculo y tolerancia.
Expected speedup...	El helper ya aprobó el valor, pero no alcanzó el umbral temporal.
Excepción o error javac	La prueba no terminó normalmente; revisar la traza o compilación.

Qué demuestra aprobar y qué requiere revisión

El registro demuestra aprobación para los casos y el entorno ejecutados. No prueba una aceleración universal ni corrección para toda entrada. El uso de streams paralelos, la ausencia de ciclos y el aislamiento de los acumuladores también se revisan en el código.

Los umbrales temporales pueden variar por JIT, recolección de basura y carga del equipo. El reloj de milisegundos puede medir intervalos de cero, sin protección explícita del cociente en los helpers. No se cambiaron los tests originales ni se presenta una ejecución temporal como garantía de rendimiento.